Group 5 Barrelfish Final Report

Chenyi Yang, Shane Nelson, Sahil Gupta

June 4, 2026

Contents

M4: Core-Local Message Passing (LMP) **2**
Architecture Overview . . . 2
Client-Side . . . 2
Server-Side . . . 3
Shared Memory for Large Messages . . . 4
Optimizations and Performance Evaluation . . . 4
Performance Goals and Optimizations . . . 4
Performance Evaluation . . . 5
Development Process . . . 7

M5: Multicore **8**
Architecture Overview . . . 8
Initial URPC Frame . . . 8
Booting Another Core . . . 9
Multicore Memory Management . . . 9
Suspend, Resume, Shutdown, and Reboot . . . 10
Development Process . . . 10

M6: User-Level Message Passing (UMP) **11**
Architecture Overview . . . 11
Initializing UMP Channels . . . 11
Memory Layout of UMP Channel . . . 12
Handling Incoming Messages . . . 12
Requestor . . . 13
Receiver . . . 13
Concurrent UMP Operations . . . 14
Extension For Shell . . . 15
API Extension . . . 15
Arbitrary Channels . . . 15
Development Process . . . 15

M4: Core-Local Message Passing

Author: Shane Nelson

The goal of this milestone is to enable threads to access services such as process management and memory allocation via intra-core remote procedure calls.

Architecture Overview

Our M4 architecture is split cleanly between our client side request API contained in `include/aos/aos_rpc.h` and our server side handlers contained in `usr/init/rpc_server.c`. This section will focus on only the content in these files relevant for LMP, there are many additional details in the source for these files, but these pertain to M6 and as such are not discussed here. Figure 1 below is a diagram illustrating the overall LMP subsystem architecture.

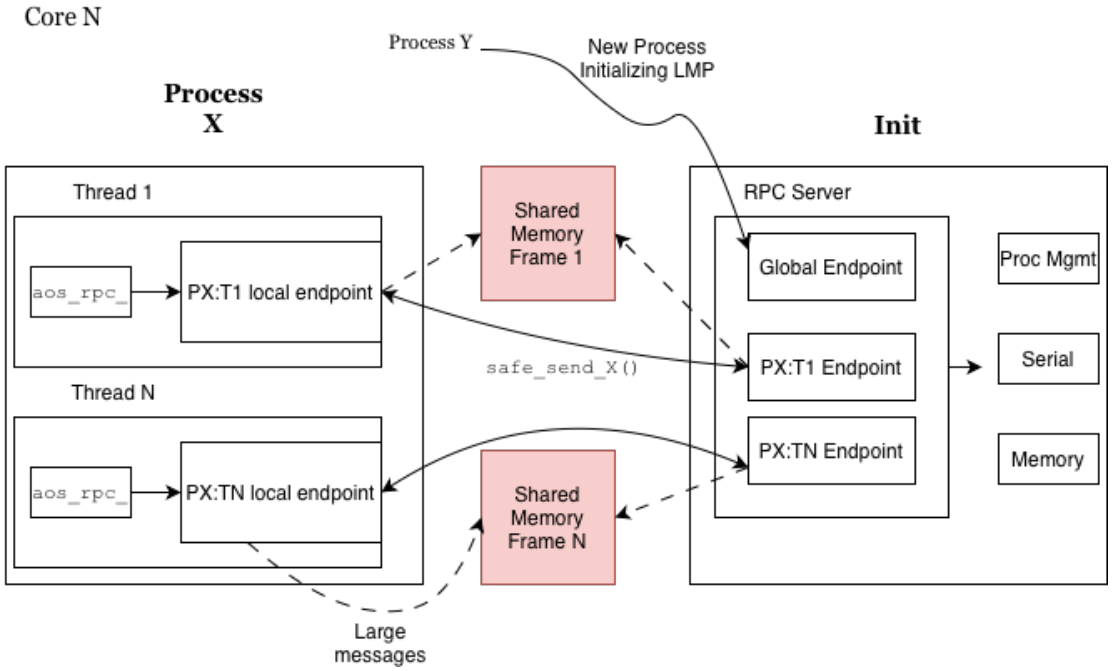


Figure 1: Overall LMP subsystem architecture.

Client-Side

We choose to implement per-thread lmp channels in order to increase the available concurrency in our system. This allows any thread to wait on a process without blocking the progress of other threads, and a similar optimization was implemented by Linux (this behavior can be achieved by passing `__WNOTHREAD` to `waitid`). This also has the benefit of making synchronization trivial but has the down-side of added initialization complexity.

Thread-Local State

Each thread keeps a `struct aos_rpc` which stores metadata about the channel, such as the channel itself and the data pertaining to shared memory. Each thread also keeps a distinct waitset which allows the underlying system to correctly wakeup threads when messages have arrived on their channel. The exposed `get_X_channel()` and `get_default_waitset()` functions are wired to use this thread-local state.

Channel Initialization

Each thread bootstraps its channel lazily on first use via a hook registered with the threading library. The initialization follows a two-step handshake. First the thread creates a temporary `lmp_chan` aimed at `cap_initep`, init's well-known global endpoint, creates a new local endpoint, and sends an `AOS_RPC_MSG_HANDSHAKE` message carrying its own local endpoint capability. Init responds by allocating a dedicated per-thread channel and replying with that channel's endpoint capability. The thread then replaces its `remote_cap` with this new per-thread endpoint, and all subsequent communication bypasses the global endpoint entirely.

Synchronous Send/Receive

All RPCs follow a strict synchronous pattern, the client sends a tagged message via `safe_sendN()` (a `lmp_chan_send()` wrapper which transparently retries on a transient error), then blocks on `recv_msg()` until a reply arrives. `recv_msg()` registers a one-shot callback on the thread's private waitset and spins on `event_dispatch()` until the callback fires and marks the slot as done. Because each thread has its own waitset and dedicated channel, threads never race to receive each other's replies.

Server-Side

In order to facilitate per-thread channels our server maintains multiple LMP endpoints. One endpoint is a dedicated global listening endpoint which handles the initialization of new channels. The server also maintains one endpoint per user process thread, created after handshaking.

Handshake Protocol

Client processes are initialized with a capability `cap_selfep` when they are initially dispatched. This capability points to the global endpoint. The client mints a new endpoint and sends it to init over the listening channel. This channel is set up to only enter in `init_handshake_handler`. Once in the handler the server allocates a new local endpoint to communicate with the child using the endpoint it receives, creates an lmp channel out of it and then sends this newly created endpoint back to the client in a `AOS_RPC_MSG_ACK` tagged message. The server then creates a `struct client_state` which stores the per-client metadata such as the `lmp_chan` and information pertaining to the shared memory frame used for large messages. Finally the server registers another function `init_recv_handler` (the general purpose entry) as the entry point for the newly created endpoint, and provides a closure with the created `struct client_state`. Now both sides can communicate on the dedicated channel.

General Purpose Serving

When a message is received on a dedicated channel, the system calls `init_recv_handler` and provides the closure created in the handshake. This function reads the message type from the first byte of the message and calls the correct handler for the desired request type. The handler which is called is determined using a table mapping message types (defined in `aos_rpc.h`) to function pointers. Once in the handler, the server will execute the desired command locally, and send the client back either an acknowledgement message or the result of the RPC.

Special Case: `aos_rpc_proc_wait()`

When a thread wants to wait on a process, it must be notified via RPC when the process exits. To facilitate this, the process management function `prod_mgmt_register_wait()` maintains a linked list of `struct waiter_node` for each process which holds metadata about the client channel. When a client calls the wait rpc function, its channel is added to this list and it is not sent a response. This blocks the thread because it is waiting in a synchronous receive loop which blocks it until a message is received on the channel. When another thread (or `libc_exit`) terminates the waitee process, `proc_mgmt_terminated` is called. This function walks the process's `waiter_node` linked list and sends an rpc message to the channel stored therein. This brings the thread out of its receive loop and unblocks it.

Shared Memory for Large Messages

In order to reduce the latency for large messages, we implement a shared memory frame to pass such messages. Without shared memory we would be forced to break large messages into cacheline-sized messages and send them one by one. This likely has large latency implications (although we did not explore testing this). To implement this lazily, before a client sends a request which requires more than one cacheline it undergoes a shared memory handshake process with init as follows:

1. Client tries to request an operation that requires a request or response larger than one cacheline and sees that `struct aos_rpc.shm_vaddr` is NULL.
2. Client allocates a physical memory frame.
3. Client maps this frame into its virtual memory space and updates the `struct aos_rpc.shm_vaddr` and `struct aos_rpc.shm_size` fields.
4. Client sends frame capability to init in a `SHMEM_SETUP` message.
5. Init maps the capability into its virtual address space and updates the corresponding `struct client_state` fields for the client.
6. Init sends back a `SHMEM_ACK` message.
7. Client proceeds to whichever operation was in progress.

Optimizations and Performance Evaluation

Performance Goals and Optimizations

Our performance goals for our optimizations of milestone 4 were (1) reduce the latency of multi-threaded RPC, and (2) reduce the memory usage of our shared-memory approach.

1. Latency Reduction for Multithreaded LMP

We anticipate multithreaded workloads that make heavy usage of the local RPC infrastructure to perform very poorly using a naive mutual-exclusion locking based approach to synchronization. For this reason we introduced per-thread LMP channels which allows for greater concurrency in the face of load. The implementation of this design is already discussed at length above, see Client-Side for details. We test the efficacy of this optimization below, see Performance Optimization.

2. Memory Usage Reduction of Large Message Infrastructure

One of the main drawbacks of the prior optimization is the memory usage due to each channel having a frame allocated for shared memory communication. This memory usage obviously increases when the number of channels increases, as it does in our prior optimization from one per process to one per thread. In a commercial system running many thousands of user-level threads, this presents a serious problem. To get around this we lazily allocate shared memory frames only when

an operation that needs a large message cannot fit the message in a certain (tunable) number of cachelines. This means that only a small subset of threads (those using RPC to pass large strings or those calling "ps" frequently) ever get a shared physical memory frame allocated. Some further optimizations would be to intelligently deallocate frames when not in use, perhaps using a variant of our `aos_rpc` API which tells the system to reclaim the frame after receipt of the data, and to make a variant of the API which allows the system to run in "memory-constrained" mode which will opt to send cachelines message by message rather than using any shared memory. We do not implement these optimizations, and importantly, due to time have not implemented memory-usage benchmarking to judge the performance of the optimization we did implement.

Performance Evaluation

Microbenchmarks

To facilitate comparing a non-optimized LMP implementation to our optimized implementation, we created three microbenchmarks:

1. **RTT: Round Trip Time.**

   This benchmark is designed to test the round trip cycle latency for a simple `aos_rpc_send_number()` operation. It is single threaded and is initially run 20 times to warm up microarchitectural structures, then 200 times each collecting a cycle latency measurement.

2. **Stress: Shared Memory RPC Stress Test.**

   This benchmark measures RPC latency under high multithreaded contention. 16 worker threads are spawned, they each run a warm-up phase, then all at once they each try to execute 50 `aos_rpc_send_string` operations, recording the latency of each operation. Thus we measure how LMP and our shared memory approach scale under high load.

3. **MTW: MultiThreaded Wait.**

   This benchmark evaluates the effect of a long-lived blocking RPC on the throughput of concurrent RPC activity. Four worker threads simultaneously send various `aos_rpc` requests to init. When run in waiting mode, another thread first waits on a separate process, then the worker threads begin. When run in baseline mode, no such waiter exists. We expect our per-thread LMP implementation to see almost no drop off between modes, while the latency of the naive implementation should be bounded by the latency of the waitee process.

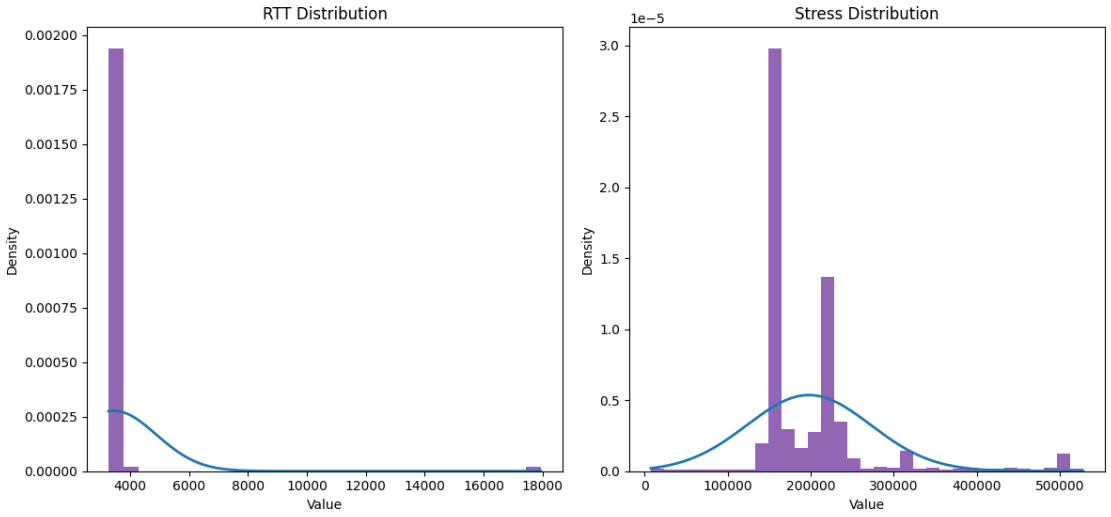
Results

We see a modest improvement in median Stress, insignificant difference in RTT, and as expected massive wins in MTW. Figure 2 displays the Stress and RTT results for the un-optimized build and Figure 3 displays the same results for our optimized build. RTT shows a significant increase in mean cycles, however this is swayed by a few outliers that are observed consistently at the end of the benchmark. Median (P50) RTT results show a slight increase for our optimized build. I believe this to be due to a slight reduction in cacheability of perthread `struct aos_rpc` compared to per-process (although this is merely a hypothesis). Interestingly, the RTT distribution for the optimized group is consistently bimodal. We observe a periodic doubling in latency for single-threaded operations, and at present we cannot explain this phenomenon, this is an important area for future investigation.

Our multithreaded stress test (Figures 2 & 3 right panel) shows that our optimized build required at mean only 78% as many cycles as the unoptimized build, and at median only 89% of cycles. Of importance is also the behavior of the tails where the unoptimized build had a significant number of latencies 2x higher than the maximum optimized latency. This explains the mean-median discrepancy. We find this to be the strongest result for our implementation as multithreaded IPC-heavy workloads are readily imaginable.

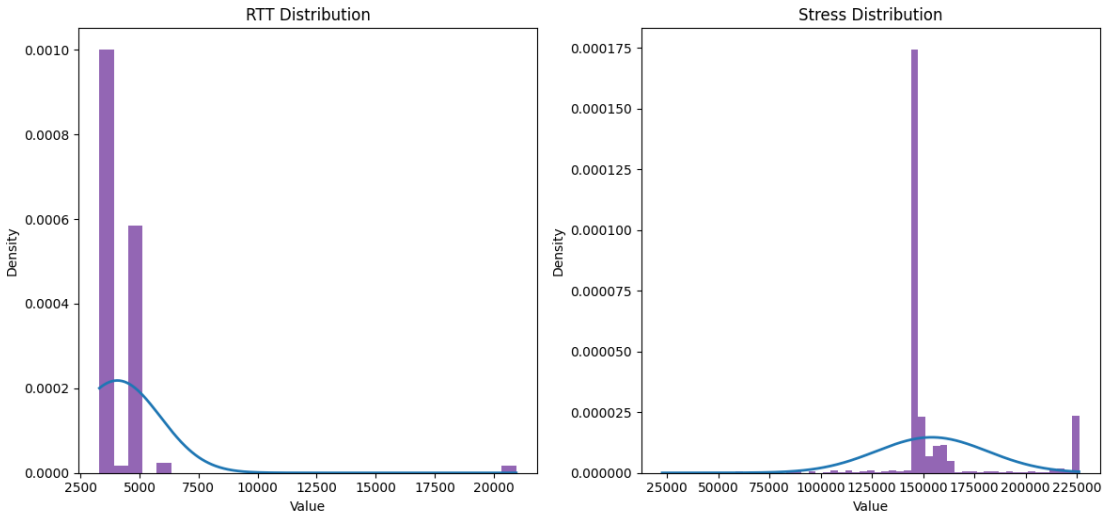
Finally, the results for our multithreaded waiter benchmark (Figure 4) reflect our initial expectations exactly. Our optimized build saw only a 26% increase in latency with a blocked waiter,

while the naive build saw a $> 1000\%$ gain in latency, bounded by the time-to-exit of the waitee process. This result reflects our initial motivation for building this optimization. It may seem heavily cherry-picked, but you could imagine an application such as a shell having such behavior.



	Mean	P50 (Median)	P90
RTT	3448.64	3286.0	3318.8
Stress	197798.64	164679.0	243952.7

Figure 2: Naive implementation benchmark results for a single-threaded number passing benchmark, labeled RTT, and a multithreaded string passing benchmark, labeled stress.



	Mean	P50 (Median)	P90
RTT	4058.51	3323.0	4759.6
Stress	154180.27	146525.0	203783.8

Figure 3: Per-thread implementation benchmark results for a single-threaded number passing benchmark, labeled RTT, and a multithreaded string passing benchmark, labeled stress. As can be seen the overhead to create a perthread channel (evident in the median RTT statistic) is negligible compared to the baseline. The per-thread implementation sees significant gains in median multithreaded latency.

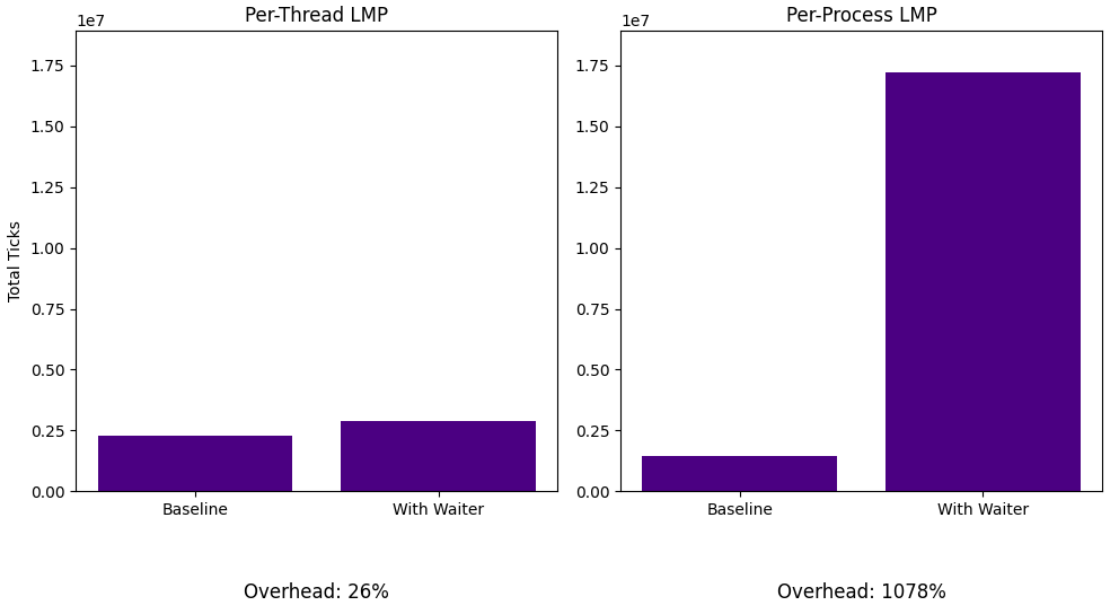


Figure 4: Performance drop-off from running a multithreaded workload (MTW) without a thread waiting on a process to finish, to running one with a thread waiting. Per process lmp channels perform $>$ 1000% worse while a thread is waiting, whereas per-thread channels see only a modest 26% cycle gain.

Development Process

What I will call phase 1 began with Sahil writing the client side state and handlers in `aos_rpc.c`/`aos_rpc.h` as well as the original initialization code in `barrelfish_initonthread()`. Allan then wrote the serverside entry point as well as wrappers for all of the per-request handlers and the implementations of the memory and serial services. Shane then finished the naive implementation by finishing the initialization, server, and client functionality, and implementing naive mutex-based synchronization and shared memory for large messages. At this point we were passing all of the `m4_grading` tests.

Phase 2 began after a conversation on Ed with Simon which motivated Shane to implement the optimizations above. This involved reworking and hardening the implementation in phase 1, as well as changing the protocol to handle per-thread LMP channels. This required touching every LMP-related file as well as the threading library and was difficult to get running, requiring debugging with Simon in office hours. Using AI tooling tests were written for the optimization, which it passed.

The final phase began when collecting data for this report, where Shane (with the help of AI tooling) reverted to an unoptimized version of the code. Shane then implemented 3 benchmarks, collected results, and wrote the M4 section of the report.

M5: Multicore

Section Author: Sahil Gupta

Architecture Overview

For multicore, our architecture is quite simple and straightforward. Cores are brought up by invoking the correct PSCI functions through the kernel (the same method is used for shutdown, reset, and reboot, as discussed later). Communication between cores for this specific milestone is handled through the `initial_urpc` struct, which lives in main memory. The relevant code for this milestone can be found in the following files:

- `usr/init/coreboot.c`
- `usr/init/coreboot.h`
- `usr/init/main.c`
- `usr/init/mem_alloc.c`
- `usr/init/mem_alloc.h`

Bonus work is discussed in the Suspend, Resume, Shutdown, and Reboot section. Below is a simple diagram outlining the process of booting the different cores and communicating with them.

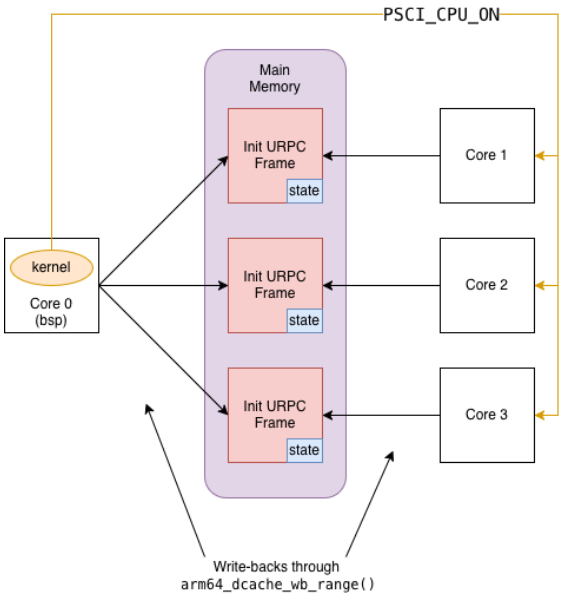


Figure 5: Multicore Architecture

Initial URPC Frame

Before discussing any of the code written for this milestone, or any of the functionality, we need to discuss the core struct for communicating with the booted cores, which contains some new fields.

```
struct initial_urpc {
    genpaddr_t            bootinfo_frame_base;
    gensize_t             bootinfo_frame_bytes;
    genpaddr_t            mmstr_frame_base;
    gensize_t             mmstr_frame_bytes;
    genpaddr_t            ram_base;
    gensize_t             ram_bytes;
    volatile corestate_t  state;
    uint32_t              active_cores_mask;
    struct {
        genpaddr_t base;
        gensize_t  bytes;
    } peer_rings[COREBOOT_MAX_CORES];
};
```

The fields `active_cores_mask` and `peer_rings` are used for User-Level Message Passing and can be ignored for now. The rest of the fields are used to boot up different cores, pass memory from the BSP core, and communicate the state of the booted core.

`bootinfo_frame_base`, `bootinfo_frame_bytes` Used to pass the booted core information about the `bootinfo` struct so it can be forged on the booted core. The `bootinfo` struct contains memory layout information crucial for the core to allocate memory.

`mmstr_frame_base`, `mmstr_frame_bytes` Passed to the booted core to forge the relevant capability, used to discover module names for processes spawned on that core.

`ram_base`, `ram_bytes` A region of RAM donated from the BSP core to give the newly spawned core memory to work with.

`state` A synchronization flag with several states, modifiable by either the BSP core or the spawned core to communicate boot progress.

Booting Another Core

Now we can talk about the process of spawning a new core. The following steps are executed by the BSP core at startup, which invokes `coreboot_boot_core` on every core (we support booting every core on the Toradex board).

1. Allocate and fill in the Initial URPC frame, then flush it to main memory so the booted core can see it
2. Locate the correct binaries for the CPU driver, boot driver, and the `init` program
3. Load the relevant ELFs and relocate them
4. Allocate a new KCB and fill in the relevant fields
5. Call `invoke_monitor_spawn_core`, which in turn invokes `PSCI_CPU_ON` through the kernel to spawn the core
6. Wait for the spawned core to signal it is ready

On the spawned core, execution begins in the `app_main` function in `usr/init/main.c`, which forges the correct capabilities and signals readiness to the BSP core by setting the `state` field.

Multicore Memory Management

Due to the design of Barrelfish, managing memory across multiple cores is non-trivial, as each core must know what memory it is responsible for. In our design, the BSP core carves out a fixed region of memory to hand over to the spawned core, using the `ram_*` fields of the `initial_urpc` struct and `ram_alloc`. We fix the size of this region at 128 MiB which is large enough for the new core to boot, but small enough not to starve the BSP core. We arrived at this value through empirical testing, and note that other values may be more appropriate depending on the multicore workload.

Suspend, Resume, Shutdown, and Reboot

In this milestone, we support the ability to suspend, resume, shutdown, and reboot a core (though reboot is not fully functional), implemented primarily through the `initial_urpc` struct. First, we need to revisit the `state` field. Its type is shown below:

```
typedef enum corestate {
    CORE_STATE_UNKNOWN,
    CORE_STATE_OFF,
    CORE_STATE_RUNNING,
    CORE_STATE_SLEEPING,
} corestate_t;
```

When the BSP core spawns a new core, the field is initialized to `CORE_STATE_UNKNOWN` and the newly spawned core then sets it to `CORE_STATE_RUNNING` once it is ready. This pattern lets a remote core detect when its state needs to change based on a signal from another core.

Suspend and shutdown are the simplest functions. Since equivalent PSCI functions exist, we create monitor entry points into the kernel and invoke them directly. When the BSP core instructs a remote core to suspend or shut down, it updates the `state` field in the URPC struct (`CORE_STATE_SLEEPING` for suspend, `CORE_STATE_OFF` for shutdown). The remote core polls this field in `app_main` and acts accordingly. This same signaling pattern is used for the other lifecycle functions.

For resume, there is no direct PSCI function. According to the PSCI specification, a suspended core requires a wakeup event to resume. We use a Software Generated Interrupt (SGI) as the wakeup event, implemented via a monitor syscall that invokes `gic_raise_softirq` on the suspended core from the kernel.

For reboot, we do not support a core rebooting itself and the only supported path is the BSP core rebooting a remote core. We accomplish this by first signaling the remote core to shut down, then invoking `coreboot_boot_core` on the same core ID.

Development Process

This milestone began with implementing the main functionality in `coreboot_boot_core` in `usr/init/coreboot.c`, where we allocated all the necessary data structures and set up the shared URPC frame. In this part, we also enabled booting all cores on the Toradex board by modifying `usr/init/main.c`.

We then implemented memory allocation on remote cores, which involved allocating a large chunk of memory through `ram_alloc` and passing it through the URPC frame. We also needed a way to initialize the allocator from a forged capability, which required us to implement `initialize_ram_alloc_from_cap` in `usr/init/mem_alloc.c`.

Finally, we implemented all the core lifecycle functions and wrote sample test cases to demonstrate the functionality.

M6: User-Level Message Passing

Author: Shane Nelson

The goal of this milestone is to enable inter-core communication between processes.

Architecture Overview

The UMP subsystem connects the init processes of each core via a shared-memory frame treated as two circular ring buffers. Processes always communicate with their local init using LMP. When a process issues an `aos_rpc` call that targets a service on a remote core, local init intercepts the LMP message and forwards it over UMP to the appropriate remote init, hence all messages require an init interception. This is intended as a simplification, but also is logical because all services in the API are served by init. We extend this original API to enable arbitrary process-to-process piping as required by a shell. The Archicture Overview section only discusses the infrastructure needed to implement the original API, see Extension for Shell for discussion on the extension. Figure 6 below illustrates the base UMP subsystem.

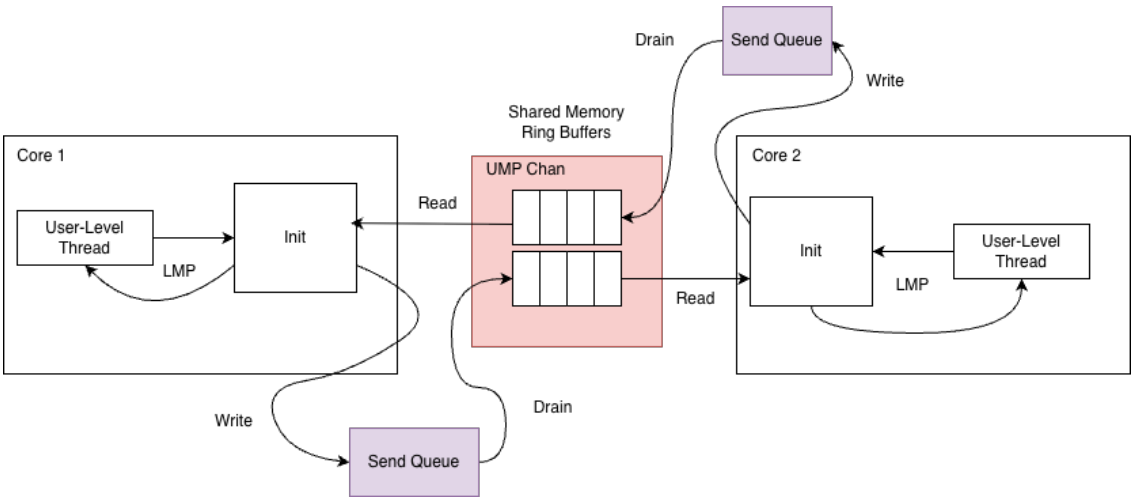


Figure 6: Abstracted view of the RPC system.

Initializing UMP Channels

Before booting any APP cores, BSP allocates a frame for each pairwise core-to-core UMP channel and stores the physical address of each frame. When booting an APP core, BSP inserts the address of each of the APP core's UMP frames (one for every peer core) in the `initial_urpc` struct along with information about which cores are online. Inside the APP core's initialization process, it maps each of these frames into its virtual address space and initializes the channels for each core that is currently running. BSP then sends a `AOS_RPC_MSG_CORE_UP` over UMP to each APP core that was previously running, telling it that a new core is online. These APP cores initialize their end of the UMP channel with the new core. As part of the initialization process, each side's init

sets a deffered timer, which periodically allows it to check for received UMP messages. At this point both sides have initialized the channel and it is ready for use.

Memory Layout of UMP Channel

A UMP channel is a shared-memory frame divided into two circular ring buffers. Each side of the connection writes into one buffer and reads from the other. Each buffer is a sequence of cache-line-sized *slots* with domains communicating by writing and reading slots. We rely on the MOESI cache coherence policy to synchronize writes and reads to these slots.

Because the ring buffer slots are a finite shared resource, a send may fail if the buffer is full requiring the sender to block. To decouple *logical* sends from *physical* availability of ring slots, each channel maintains a dynamically-allocated send queue (linked list) of messages waiting to be sent. When a physical send would block, the message is instead appended to this queue. A periodic timer drains the queue by calling `drain_send_queue`, which flushes enqueued entries into the ring as slots become free. From the caller's perspective, a send always succeeds immediately. The actual placement into shared memory is deferred transparently, allowing the sender to proceed handling more LMP requests.

For most RPC operations, all data fits in a single slot. However, some requests and replies exceed one cache line and must be split across multiple slots. These *multi-slot* messages consist of a *header slot* followed by one or more *continuation slots*. The header encodes the message tag, the total number of words in the message, and the first five data words. Continuation slots carry the remaining data words. In one byte the message tag encodes the RPC operation and the message's role in the protocol: request header, reply header, request continuation, or reply continuation

To reassemble multi-slot messages before dispatching them, we introduce two auxiliary tables. `pending_recv` is used by the receiver to buffer and assemble incoming multi-slot requests until all continuation slots have arrived. `pending_replies` is the symmetric table used by the requestor to buffer incoming multi-slot replies. These tables are statically sized and allocated. Importantly, the message tag also encodes the index into the tables (which are symmetrically indexed) as a means of uniquely identifying to which transaction a particular message belongs. This ensures that messages are assembled and dispatched atomically even in the presence of non-atomic sends (messages for different operations may interleave). When a message is fully constructed, it is "dispatched", the actual behavior of the dispatch depends on if the domain is the original requestor (i.e. write end) or the receiver (i.e. read end) for the message.

Handling Incoming Messages

When the timer set on init fires, execution is started in the timer handler function. The timer handler function, `ump_poll_handler()`, loops through all UMP channels and processes any messages which may have been received. Based on the message type, the handler does the following:

Request Header:

1. If the entire message fits in the header, immediately dispatch into the correct header based on the request type.
2. If the entire message does not fit in the header, allocate a slot in the `pending_recv` table storing the size of the message, how many bytes weve received so far and the type of the request.

Request Continuation:

1. If the message is now complete, free the `pending_recv` entry and dispatch.
2. Otherwise update how many bytes we still need to receive.

Reply Header:

1. If the entire message fits in the header, immediately respond to the LMP client. Deallocate the previously allocated `pending_replies` entry. (See Requestor for discussion on how this was allocated).
2. If the entire message does not fit in the header, update the `pending_replies` entry

Reply Continuation:

1. If the reply is now complete, respond to the LMP client. Deallocate the previously allocated `pending_replies` entry.
2. Otherwise update how many bytes we still need to receive.

Requestor

The requestor in the UMP protocol is the init domain which initiates an RPC call on behalf of a process on its core. An init may act as both Requestor and Receiver for different messages concurrently, but each message has one requestor and receiver.

When init receives an LMP request, it determines the target core. For `aos_rpc_proc_spawn_with_cmdline()`, the target core is specified as an argument. For other process management functions, the process ID of the target process is supplied as an argument. We encode the target core in the PID. If a request targets a remote core init does as follows:

1. Ensure that the target core is running, if not return an error.
2. Allocate an entry in `pending_replies` to buffer any replies the remote core may send.
3. If there are no entries available in the buffer, send a transient error to the client. The client will then prompt the kernel to execute a context switch. When the client is rescheduled it retries the operation. All of this occurs transparently so the programmer must not explicitly handle such errors.
4. Parse LMP arguments into required header and cont messages.
5. Enqueue messages into the send queue and attempt to drain the queue, sending the messages on the UMP channel to the remote core.
6. Handle other business while the LMP client maintains blocked waiting for a reply.

When the timer fires, `ump_poll_handler` checks for replies. If the reply is ready the requestor translates the UMP reply into an LMP reply. The client LMP channel is stored in the `pending_replies` entry, thus init knows where to send the LMP reply.

Facilitating Global Operations

There are certain operations such as kill all and ps which require init to communicate with the init on every core rather than only one core's init. For such operations we find a `pending_replies` entry for each core that is online, making note that the entry is for a global operation. We then send UMP requests to each UMP channel in the exact same method as above. In order to track the incoming replies we introduce another static datastructure `ump_fanout`. This struct holds information used to compile the final result as well as information about which cores still need to respond. Once all responses have been gathered we compile the results and reply to the client via the lmp channel stored in `ump_fanout`. There is only one `struct ump_fanout` for each init image so each core can only have one outstanding global operation at any point.

Receiver

The receiver is much simpler than the requestor. Once the entire request is compiled via `ump_poll_handler` the correct handler is called for the required rpc operation. We pass the handler a newly created `client_state` struct which encodes that this is a UMP request. Once in the handler we execute the rpc command locally. We then assemble the UMP reply into a header and any necessary continuation messages and attempt to send them over the UMP channel.

Concurrent UMP Operations

The `pending_replies` buffer mentioned above allows our system to handle multiple concurrent UMP transactions. A naive implementation may elect not to have such a buffer and instead only allow one outstanding UMP transaction at any point in time. This has the benefit of being simple to implement and also implicitly ensuring that send and receive pairs are atomic (a sender can be ensured that the response it receives will be its response). To test the performance benefits of allowing concurrent UMP transactions, we wrote a small microbenchmark. This benchmark spawns 16 threads, each performing one cross-core rpc operation, and records the total time for all threads to complete (this runs 50 times to get a large enough sample). We simulate the naive approach with a one-entry `pending_replies`. Figure 7 below shows the results of this benchmark.

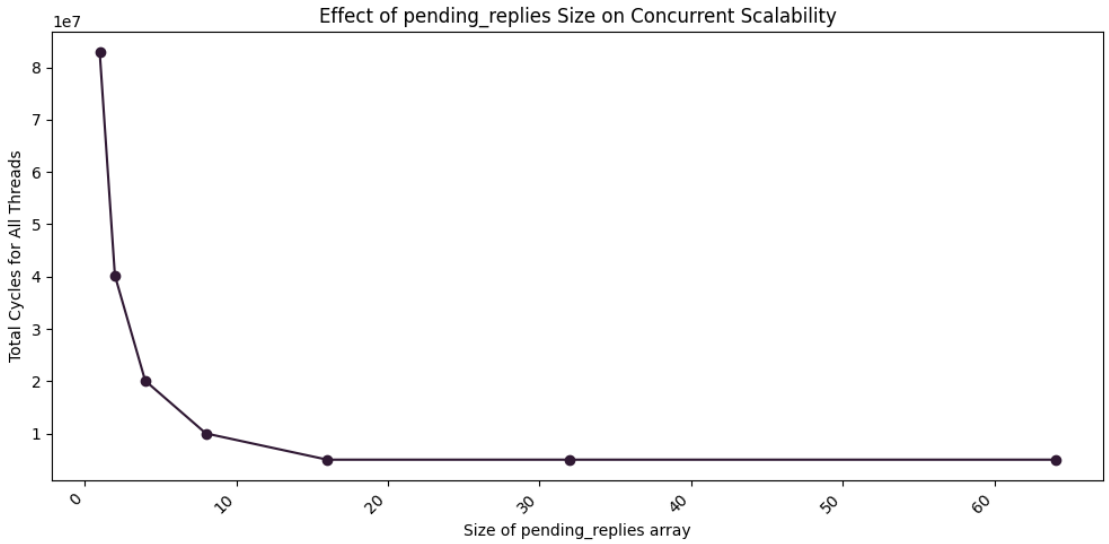


Figure 7: Performance scales linearly with the number of slots in `pending_replies`, until there are enough slots for each thread at which point there are no additional returns.

The results are exactly as expected, performance scales linearly for each additional `pending_replies` slot added. Once there is a slot for every thread the benchmark spawns [corollary: outstanding UMP request] there are no more performance gains.

This leads to an important performance invariant, in order to have optimal concurrent performance, the UMP service must be able to have a `pending_replies` and `pending_recv` (see below) slot for every concurrent UMP operation. Because LMP is synchronous, this means that there need only be a `pending_replies` slot for each thread for maximum performace under high contention. We do not implement this because depending on implementation it would impose necessary communication between init domains on how many threads there are or best case requires increased complexity in tracking operations. We hypothesize this maximal approach is not necessary because most workloads do not have every thread simultaneously communicating cross-core. We we choose an approach (64 slots) which allows for enough concurrency without complicating the protocol and design. Future work may elect to implement a dynamic allocation scheme or otherwise allow the threading library (and perhaps user) to scale the available concurrency in the system.

Extension For Shell

API Extension

Arbitrary Channels

Development Process